1st project - Neural Networks

Κουδουνης Βασιλειος
10739

Χειμερινο εξαμηνο 2024

1 Introduction

In this project I managed to build from scratch a MLP neural network which is trained with the back-propagation algorirthm. I used the CIFAR-10 dataset as my database to train and classify the data. I worked in google colab using python.

2 Description of the code

The code starts with the downloading of our data and some function to load each CIFAR-10 batch file, together with some necessary reshapes of the arrays in which the data are loaded, for easier usage later in our neural network.

Afterwards, I implemented a class called Layer, which has the methods that a layer needs in a neural network.
1. The init method that sets the weights and biases as well as the activation function of the layer once it is created. The initial weights values are random because random initialization breaks the symmetry by giving each neuron unique weights, enabling them to learn different features during training, whereas biases are being set to zero because they are less prone to symmetry-breaking issues than weights.
2. The forward method that "passes" the information to the next layer using the known relationship between input and output: output = weights * input + bias. For the hidden layers the activation function that is used is the relu, whereas for the output layer the softmax function is used.
3. Last but not least we have the backward method, which propagates the error from the output layer back until the input layer. With the help of the well-known chain rule of calculus we are able to calculate step by step the gradient of the loss with respect to the layer's input. Gradients with respect to the weights, biases and the activation function will be used to update these parameters.

In order to be able to control the parameters of the neural network, I implemented the class MLP where all the network layers are being defined(in terms of input,output neurons and activation function). In this class there are also the forward and backward methods to spark the training process of the network.

2.1 Brief mention for some key functions

1. The cross-entropy-loss function measures the difference between the true labels and the predicted probability distributions output. For a single data point x_i with a true label y_i and predicted probabilities p_i , the cross-entropy loss is: $L = -Sum_{i=1}^{C}(yi * log(pi))$ and it quantifies how well the predicted probabilities match the true labels. This way of computing the loss is quite efficient, because it assigns a higher penalty for confident wrong predictions(e.g., predicting 0.99 for the wrong class). Moreover, it does suit with the softmax activation function for the output layer, since we are operating with probabilities.

2. The calculate-accuracy function estimates the accuracy of our model by comparing the predicting labels of the classes with the actually true one.

3. The preprocess-data function normalizes the pixel values of the images, by scaling them between 0 and 1, and reshapes the images so that they are in the appropriate form to "enter" our MLP.

4. A very important detail is the one-hot-encode function that converts class labels into a one-hot encoded format. For example for a label 4 (that refers to the 4th class) it will be turned into a vector of length(number of classes=10) with zeros everywhere except the 4th element which will be 1. This format is essential for loss-function compatibility and makes it easier to perform vectorized operations on the labels.

2.2 into the training loop

After the definitions of all the necessary functions, we reach the training loop. We set the hyperparameters like epochs, batch-sizes and learning rate, accordingly with our desired values each time and we head into the shuffle of the training data for each epoch. We go through all the batches and compute the loss, after which we calculate the training and test accuracy using the training and test data for each. I have tested several combinations of the hyperparameters, and i will present the results together with some conclusions.

3 Display of the results

3.1 2 hidden layers

learning rate = 0.001, epochs=60, batch-size=64 — neurons = 512/256/128 —training time = 90 minutes

Epoch	Training Loss	Training Accuracy (%)	Test Accuracy (%)
1	2.2999	0.1330	0.1315
5	1.8008	0.3531	0.3483
10	1.5473	0.4823	0.4749
15	1.387	0.5178	0.4953
20	1.2574	0.5324	0.4863
25	1.1431	0.5969	0.5125
30	1.0347	0.6256	0.5140
40	0.8231	0.7270	0.5342
50	0.6343	0.8083	0.5325
60	0.4670	0.8516	0.5350

learning rate = 0.001 — epochs=70 — batch-size=64 — neurons = 1024/256/64 — training time = 96 minutes

Epoch	Training Loss	Training Accuracy (%)	Test Accuracy (%)
1	2.2916	0.1871	0.1857
5	1.7458	0.4029	0.4117
10	1.4790	0.4684	0.4494
15	1.3170	0.5470	0.5106
20	1.1931	0.5882	0.5184
25	1.0689	0.6448	0.5356
30	0.9415	0.6786	0.5328
40	0.7107	0.7803	0.5330
50	0.5161	0.8044	0.5263
60	0.3930	0.8880	0.5337
70	0.2821	0.9225	0.5372

learning rate = 0.001, epochs=70, batch-size=64—neurons = 256/128/64—training time=85 minutes

Epoch	Training Loss	Training Accuracy (%)	Test Accuracy (%)
1	2.3058	0.1000	0.1000
5	1.9210	0.2679	0.2635
10	1.6039	0.4163	0.4136
15	1.4187	0.5058	0.4817
20	1.2972	0.5244	0.4823
25	1.2032	0.5804	0.5100
30	1.1191	0.6326	0.5361
40	0.9680	0.6891	0.5447
50	0.8396	0.7313	0.5392
60	0.7265	0.7655	0.5367
70	0.6361	0.8028	0.5243

Metric	Dataset 1	Dataset 2	Dataset 3
Peak Test Accuracy	53.50% (Epoch 60)	53.72% (Epoch 70)	**54.47% (Epoch 40)**
Overfitting Start	Epoch 30	Epoch 25–30	**Epoch 40**
Training-Test Gap	~31.66% (at Epoch 60)	~38.53% (at Epoch 70)	**22.88% (at Epoch 60)**
Test Accuracy Plateau	Early (Epoch 40)	Late (Epoch 70)	Moderate (Epoch 40)

Figure 1: 1 = 512/256/128 — 2 = 1024/256/64 — 3 = 256/128/64

So, Dataset 3 with 256/128/64 number of neurons is the best overall, because:
1. It achieves the highest peak test accuracy (54.47%).
2. Overfitting starts later and is less severe.
3. Test accuracy peaks early, requiring less training time.

Varying batch-sizes

learning rate = 0.001, epochs=50, batch-size=128—neurons = 256/128/64—training time = 70 minutes

Epoch	Training Loss	Training Accuracy (%)	Test Accuracy (%)
1	2.3087	0.1000	0.1000
5	1.9345	0.3103	0.3087
10	1.6542	0.4296	0.4289
15	1.4873	0.4911	0.4739
20	1.3704	0.5388	0.5115
25	1.2753	0.5584	0.5073
30	1.1946	0.5996	0.5312
40	1.0472	0.6590	0.5288
50	0.9186	0.7117	0.5388

learning rate = 0.001, epochs=50, batch-size=32—neurons = 256/128/64—training time = 85 minutes

Epoch	Training Loss	Training Accuracy (%)	Test Accuracy (%)
1	2.3043	0.1000	0.1000
5	1.9007	0.2760	0.2771
10	1.5650	0.4560	0.4431
15	1.3869	0.5054	0.4800
20	1.2650	0.5560	0.5095
25	1.1692	0.5910	0.5197
30	1.0818	0.6303	0.5383
40	0.9310	0.6786	0.5337
50	0.7999	0.7204	0.5343

Metric	Configuration 1	Configuration 2	Configuration 3
Peak Test Accuracy	53.88% (epoch 50)	**54.47% (epoch 40)**	53.43% (epoch 50)
Training Accuracy	71.17% (epoch 50)	**73.13% (epoch 50)**	72.04% (epoch 50)
Training Loss	0.9186 (epoch 50)	0.8396 (epoch 50)	**0.7999 (epoch 50)**
Overfitting Gap	~17.29%	**~19.21%**	~18.61%
Convergence Speed	Peak around epoch 50	Peak around epoch 40	**Peak around epoch 30**

Figure 2: 1 =batch 128, 2 = batch 64, 3 = batch 32

Selecting Batch size = 64 is the strongest in terms of test accuracy but shows the most overfitting.
Selecting Batch size = 32 balances training efficiency, lower training loss, and reduced overfitting, though its test accuracy slightly lags behind Case 2.
Selecting Batch size = 128 is the simplest and most efficient, converging quickly but slightly underperforming case 3.
Using early stopping at about 40 epochs, we can assume that for batch-size = 64, the network is somewhat more efficient.

learning rate = 0.01, epochs=30, batch-size=64 neurons = 256/128/64 training time = 30 minutes

Epoch	Training Loss	Training Accuracy (%)	Test Accuracy (%)
1	2.3052	0.1000	0.1000
5	2.0532	0.2459	0.2508
10	2.0757	0.2661	0.2667
15	2.3081	0.1000	0.1000
20	2.3079	0.1000	0.1000
25	2.3080	0.1000	0.1000
30	2.3080	0.1000	0.1000

As it seems the network completely fails to learn when I set the learning rate to 0.01.At the start, the model shows some progress in reducing the loss and increasing accuracy (epochs 1–10). However, by epoch 15, both training and test accuracies regress to their initial values (10% accuracy, equivalent to random guessing for CIFAR-10), and the loss remains stuck at a high value of 2.3. This suggests that the learning rate is too high, causing the optimizer to overshoot the optimal weights. The divergence at epoch 15 is consistent with exploding gradients or weights being pushed into regions where the model cannot recover. The gradient updates may no longer produce meaningful improvements due to excessive oscillations. The learning rate of 0.01 is too aggressive for this configuration, causing training to fail catastrophically after a few epochs.

learning rate = 0.0001, epochs = 200, batch-size=64, neurons = 256/128/64 training = 1 hour

Epoch	**Training Loss**	**Training Accuracy (%)**	**Test Accuracy (%)**
1	2.3056	0.0999	0.1001
5	2.3055	0.1000	0.1000
10	2.3055	0.1001	0.1005
15	2.3051	0.1645	0.1644
20	2.2931	0.1404	0.1399
25	2.0535	0.1922	0.1870
30	2.0020	0.2266	0.2271
40	1.8453	0.2932	0.2934
50	1.7342	0.3557	0.3554
60	1.5676	0.4375	0.4312
70	1.4369	0.4742	0.4530
80	1.3621	0.5169	0.4838
90	1.2832	0.5261	0.4742
100	1.2109	0.5641	0.4963
110	1.1412	0.57	0.4951
120	1.0786	0.6225	0.5202
130	1.0134	0.6574	0.5231
140	0.9552	0.6472	0.5083
150	0.8981	0.6800	0.5188
160	0.8493	0.7012	0.5157
170	0.8020	0.7146	0.5254
180	0.7385	0.7616	0.5278
190	0.6838	0.7619	0.5183
200	0.6390	0.7638	0.5143

In the first 10 epochs, the loss remains almost constant at 2.305, and the train/test accuracy hovers around random guess levels (10% for a 10-class problem, likely indicating the model is not learning).This stagnation is explained by the fact that extremely low learning rates cause the gradient updates to be too small to cause significant progress. Afterwards, the model appears to learn in a very slow but steady way and by epoch 100, the test accuracy reaches 50%, which is a sign that the model is learning meaningfully. After epoch 100, the improvement becomes more incremental, with both training and testing accuracy nearing a plateau by epoch 150–200. The train/test accuracies (70%/53% by epoch 150) suggest underfitting. The model hasn't yet leveraged the full capacity to learn, possibly due to the low learning rate restricting the gradient descent steps.The test accuracy closely tracks the train accuracy throughout, with only slight overfitting, which is typical for low learning rates, as updates are conservative.All in all, the learning rate of 0.0001 appears too low for efficient convergence.

3.2 1 hidden layer

learning rate = 0.001, epochs=70, batch-size=64, neurons = 256/64—training time = 45 minutes

Epoch	Training Loss	Training Accuracy (%)	Test Accuracy (%)
1	2.1385	0.2685	0.2678
5	1.6241	0.4389	0.4323
10	1.4389	0.5208	0.4947
15	1.3232	0.5508	0.5106
20	1.2356	0.5819	0.5155
25	1.1633	0.5973	0.5176
30	1.0972	0.6242	0.5232
40	0.9873	0.6776	0.5350
50	0.8820	0.7032	0.5275
60	0.7992	0.7499	0.5322
70	0.7249	0.7566	0.5181

learning rate = 0.001 — epochs=70 — batch-size=64 — neurons = 512/128 — training time: 49 minutes

Epoch	Training Loss	Training Accuracy (%)	Test Accuracy (%)
1	2.0799	0.3274	0.3235
5	1.5772	0.4228	0.4151
10	1.3919	0.5235	0.4959
15	1.2642	0.5816	0.5307
20	1.1582	0.6077	0.5359
25	1.0638	0.6553	0.5491
30	0.9722	0.6488	0.5179
40	0.8059	0.7347	0.5415
50	0.6549	0.7834	0.5329
60	0.5270	0.8165	0.5211
70	0.4295	0.8641	0.5221

learning rate = 0.001, epochs=70, batch-size=64, neurons = 1024/256— training time = 42 minutes

Epoch	Training Loss	Training Accuracy (%)	Test Accuracy (%)
1	2.0245	0.3322	0.3342
5	1.5481	0.4670	0.4580
10	1.3543	0.5155	0.4836
15	1.2199	0.5954	0.5299
20	1.0884	0.6296	0.5326
25	0.9670	0.6690	0.5311
30	0.8412	0.7117	0.5373
40	0.6107	0.8116	0.5508
50	0.4199	0.8833	0.5504
60	0.2818	0.9211	0.5523
70	0.2001	0.9144	0.5321

Some conclusions:
Dataset with 1024/256 neurons performs the best overall, achieving the lowest training loss and highest accuracies. However, it also shows signs of overfitting by the later epochs, as indicated by a widening gap between training and test accuracy.
Dataset with 512/128 neurons balances performance well, showing competitive test accuracy with less extreme overfitting than Dataset with 1024/256 neurons.
Dataset with 256/64 neurons lags in both training and test metrics, suggesting less effective learning compared to the others.

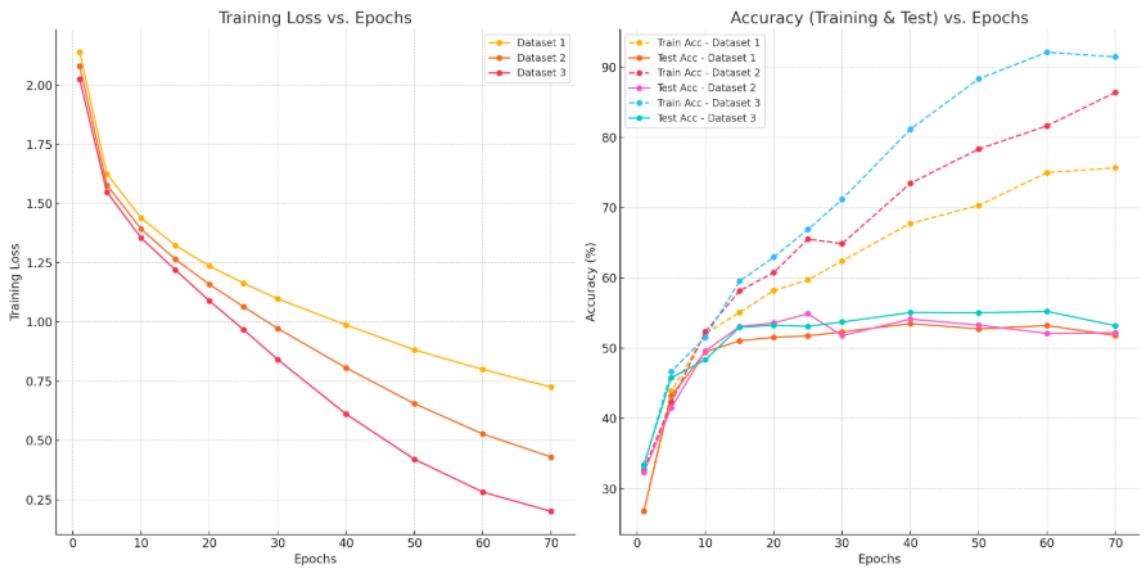


Figure 3: 1 = 256/64, 2 = 512/128, 3 = 1024/256.

Overfitting Indications:
Dataset 1: The gap between training and test accuracy grows from 0.07% (Epoch 1) to 23.85%(Epoch 70).
Dataset 2: The gap grows from 0.39% (Epoch 1) to 34.2% (Epoch 70).
Dataset 3: The gap grows from 0.20% (Epoch 1) to 38.23% (Epoch 70).
Observation: All datasets show increasing overfitting over time, with Dataset 3 overfitting the most (largest gap between training and test accuracy). We could say that overfitting in dataset 3 becomes severe around about 40 epochs, in contrast with dataset 2 that starts at around 25 epochs. So using early stopping at 40 epochs I will assume that dataset 3 is slightly better, although I don't think there are significant improvements.

Varying batch-sizes:

learning rate = 0.001, epochs=40, batch-size=128, neurons = 1024/256

Epoch	Training Loss	Training Accuracy (%)	Test Accuracy (%)
1	2.0659	0.3120	0.3130
5	1.6161	0.4401	0.4312
10	1.4269	0.5079	0.4790
15	1.2864	0.5489	0.4963
20	1.1776	0.6152	0.5371
25	1.0555	0.6635	0.5406
30	0.9352	0.6908	0.5365
40	0.7005	0.7918	0.5463

learning rate = 0.001, epochs=40, batch-size=32, neurons = 1024/256

Epoch	Training Loss	Training Accuracy (%)	Test Accuracy (%)
1	2.0075	0.3487	0.3477
5	1.5133	0.4893	0.4718
10	1.3257	0.5566	0.5155
15	1.1871	0.6047	0.5367
20	1.0563	0.6405	0.5309
25	0.9346	0.6859	0.5360
30	0.8105	0.7372	0.5481
40	0.5773	0.7962	0.5308

General Observations

The smallest batch size (32) achieves the lowest final training loss (0.5773 at epoch 40), indicating better fitting to the training data. The largest batch size (128) has the highest final training loss (0.7005 at epoch 40).The best final test accuracy is achieved with a batch size of 64, reaching 55.08% at epoch 40. Smaller batch sizes (32) lead to better training accuracy but potentially worse generalization due to overfitting.

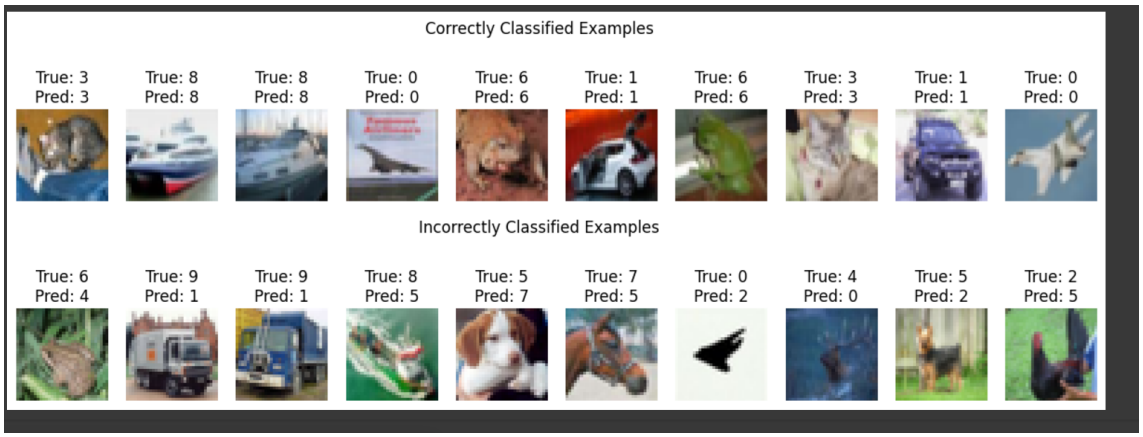
Genarally the most overfitting occurs for batch-size 32 and the least overfitting for batch-size 128.Batch size 64 strikes a good trade-off between computational efficiency and model performance, as it achieves the highest test accuracy.

learning rate = 0.0001, epochs=100, batch-size=64, neurons = 1024/256, training time = 54 minutes

Epoch	**Training Loss**	**Training Accuracy (%)**	**Test Accuracy (%)**
1	2.2954	0.1346	0.1380
5	1.9205	0.3122	0.3070
10	1.7482	0.3783	0.3772
15	1.6237	0.4260	0.4208
20	1.5233	0.4646	0.4534
25	1.4491	0.4919	0.4755
30	1.3820	0.5091	0.4987
40	1.2704	0.5608	0.5193
50	1.1777	0.5795	0.5332
60	1.0987	0.6306	0.5429
70	1.0155	0.6573	0.5489
80	0.9367	0.6689	0.5402
90	0.8537	0.6852	0.5300
100	0.7714	0.7548	0.5449
110	0.7052	0.7690	0.5451
120	0.6234	0.7940	0.5488
130	0.5558	0.8099	0.5418
140	0.4849	0.8273	0.5338
150	0.4169	0.8977	0.5504
160	0.3680	0.9023	0.5542

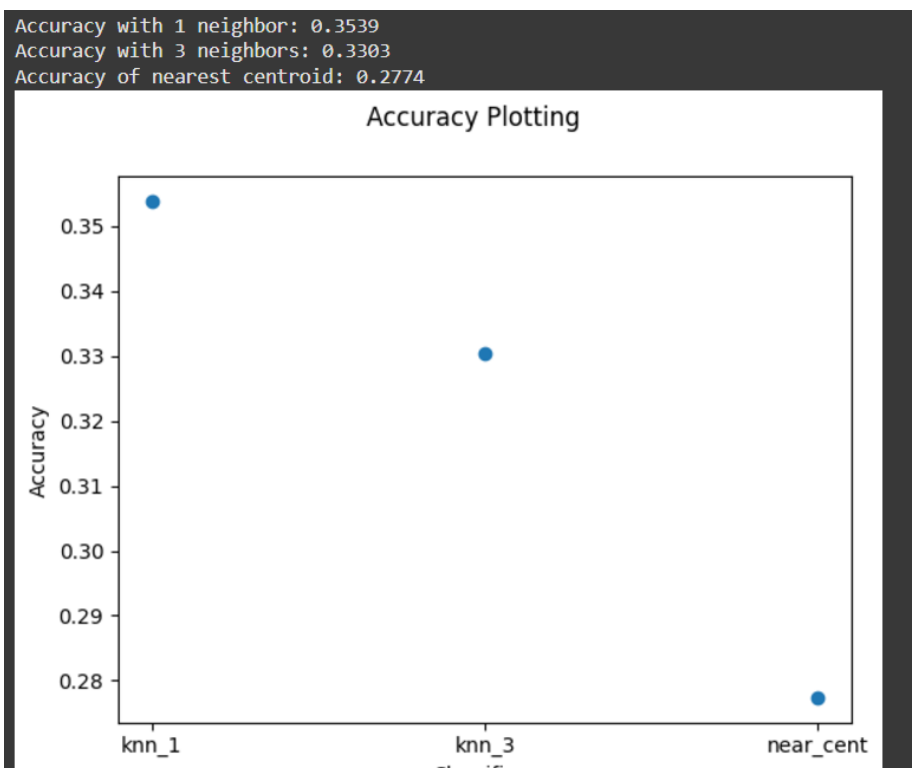
In the early epochs, a steady decrease in loss and improvement in both training and test accuracy is observed. By epoch 20, train and test accuracies improve to 45.56% and 44.68%, respectively. Loss continues to decline steadily, reaching 0.7714 by epoch 100.Training accuracy rises significantly, exceeding 75% by epoch 100, with the test accuracy stabilizing around 54-56%.The gap between training and test accuracy widens slightly, suggesting the start of mild overfitting.Training accuracy surpasses 85% (e.g., epoch 150: 89%), while test accuracy plateaus around 55-56%, showing limited generalization improvement. Again. overfitting becomes more and more apparent as the training accuracy far exceeds the test accuracy. Generally, the learning rate (0.0001) is conservative, which is reflected in the steady but slow improvement. While this prevents sudden fluctuations, it requires more epochs for convergence. The increasing divergence between training and test accuracy suggests overfitting starting around epoch 70-100.

4 Examples of correct and incorrect classifications



In this picture, we can see several examples of correct and incorrect classifications. I trained the model with one hidden layer at a learning rate of 0.001 with batch size = 64 at 60 epochs, which proved to be one of the most optimal training models. It seems that the wrong predictions are being confused with similar classes, such as horses with dogs, trucks with cars, dogs with chickens, which could be a sign that some work is being done.

5 Comparison between NN and NCC classifiers



The most efficient algorithm proved to be the 1-Nearest-Neighbour method, followed by the 3-Nearest-Neighbuors and lastly by the Nearest Centroid with the accuracies shown in the figure above. Comparing our neural network with the Nearest Neighbour Approximation and Nearest Centroid, the gain in the accuracy is evident, approximately 20% difference. This result is logical as neural networks learn abstract features through hidden layers, which often allow for better generalization across diverse datasets compared to nearest neighbor methods that rely purely on raw distance metrics. Also, it is important

to note that although the model needs more time to finally converge in its best accuracy than the other techniques, if we wanted it to reach only 27-35% it would be much faster than them, as it can reach this accuracy in early epochs, in some minutes/seconds depending on the sample. Last but not least, while nearest neighbors might work efficiently with small datasets, the computational cost becomes prohibitive as the dataset size increases. Neural networks, on the other hand, can handle larger datasets more efficiently once trained.

6 Some final conclusions

It seemed like the model behaved somewhat better at the learning rate of 0.001, batch size = 64, while the number of neurons didn't appear to influence the model significantly. Quite acceptable results were obtained with several combinations of the layers in terms of the input/output relationship.